



PDF Download  
3340531.3412136.pdf  
24 February 2026  
Total Citations: 41  
Total Downloads: 683

 Latest updates: <https://dl.acm.org/doi/10.1145/3340531.3412136>

SHORT-PAPER

## CONE-Align: Consistent Network Alignment with Proximity-Preserving Node Embedding

XIYUAN CHEN, University of Michigan, Ann Arbor, Ann Arbor, MI, United States

MARK HEIMANN, University of Michigan, Ann Arbor, Ann Arbor, MI, United States

FATEMEH VAHEDIAN, University of Michigan, Ann Arbor, Ann Arbor, MI, United States

DANAI KOUTRA, University of Michigan, Ann Arbor, Ann Arbor, MI, United States

Open Access Support provided by:

University of Michigan, Ann Arbor

Published: 19 October 2020

[Citation in BibTeX format](#)

CIKM '20: The 29th ACM International  
Conference on Information and  
Knowledge Management  
October 19 - 23, 2020  
Virtual Event, Ireland

Conference Sponsors:

SIGIR  
SIGWEB

# CONE-Align: Consistent Network Alignment with Proximity-Preserving Node Embedding

Xiyuan Chen

University of Michigan, Ann Arbor  
shinech@umich.edu

Fatemeh Vahedian

University of Michigan, Ann Arbor  
vfatemeh@umich.edu

Mark Heimann

University of Michigan, Ann Arbor  
mheimann@umich.edu

Danai Koutra

University of Michigan, Ann Arbor  
dkoutra@umich.edu

## ABSTRACT

Network alignment, the process of finding correspondences between nodes in different graphs, has many scientific and industrial applications. Existing unsupervised network alignment methods find suboptimal alignments that break up node neighborhoods, i.e. do not preserve *matched neighborhood consistency*. To improve this, we propose CONE-Align, which models intra-network proximity with node embeddings and uses them to match nodes across networks after aligning the embedding subspaces. Experiments on diverse, challenging datasets show that CONE-Align is robust and obtains 19.25% greater accuracy on average than the best-performing state-of-the-art graph alignment algorithm in highly noisy settings.

### ACM Reference Format:

Xiyuan Chen, Mark Heimann, Fatemeh Vahedian, and Danai Koutra. 2020. CONE-Align: Consistent Network Alignment with Proximity-Preserving Node Embedding. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM '20)*, October 19–23, 2020, Virtual Event, Ireland. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3340531.3412136>

## 1 INTRODUCTION

Graphs or networks are ubiquitous structures for representing complex interconnections between entities. One important graph-based data mining task is network alignment: finding correspondences between nodes in different graphs. This task has diverse, important applications, such as recommendation on social networks, protein-protein interaction analysis, and database schema matching [15].

This work is inspired by a common limitation of network alignment methods. We find that many unsupervised graph alignment approaches (e.g., FINAL [25], NetAlign [3], REGAL [14]) fail to achieve *matched neighborhood consistency*: nodes that are close in one graph are often not matched to nodes that are close in the other graph. For example, REGAL [14] matches nodes using *node embeddings* capturing each node's structural role in the network. However, neighboring nodes may not have similar structural roles, resulting

in very different embeddings that may be matched far apart in the other graph, violating matched neighborhood consistency.

To solve this problem, we propose CONE-Align for CONsistent Emboding-based Network Alignment. We learn similar node embeddings for neighboring nodes in each graph using well-known proximity-preserving node embedding methods. However, because nodes are not in proximity across graphs, these methods are transductive, and nodes in different graphs will be embedded into different subspaces. Therefore, we align the graphs' embedding subspaces, and then we can match the nodes using embedding similarity. Since neighboring nodes in each graph will have similar embeddings, they will be matched to similar parts of the other graph. Thus, we have the best of both worlds with CONE-Align: matched neighborhood consistency and cross-graph comparability.

Our contributions can be summarized as follows:

- **Insights for Network Alignment:** We define the principle of *matched neighborhood consistency*, which motivates us to use node embedding methods with a different kind of objective than what has been used for unsupervised network alignment.
- **Principled New Method:** We propose CONE-Align for unsupervised network alignment, which makes embedding subspaces for different graphs comparable, analogous to machine translation using monolingual word embeddings.
- **Rigorous Experiments:** On challenging datasets, we show that CONE-Align outperforms the strongest baseline by 19.25% on average in accuracy, as it better preserves matched neighborhood consistency. Our code is available at <https://github.com/GemsLab/CONE-Align>.

## 2 RELATED WORK

**Node Embeddings.** Node embeddings are latent feature vectors modeling relationships between nodes and/or structural characteristics, learned with various shallow and deep architectures and used for many graph mining tasks [10]. Most embedding objectives model **proximity** within a single graph: nearby nodes (e.g. neighbors sharing an edge or nodes with mutual neighbors) have similar features. For example, DeepWalk [20] and node2vec [12] perform random walks starting at each node to sample context nodes, using a shallow neural architecture to embed nodes similarly to their context. This process implicitly factorizes a node pointwise mutual information matrix, which NetMF [21] instead directly factorizes.

In contrast, **structural** embedding methods capture a node's structural role independent of its proximity to specific nodes; this independence makes embeddings comparable across graphs [14].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

CIKM '20, October 19–23, 2020, Virtual Event, Ireland

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-6859-9/20/10...\$15.00

<https://doi.org/10.1145/3340531.3412136>

For example, struc2vec [22] resembles DeepWalk and node2vec but performs random walks on an auxiliary structural similarity graph. xNetMF embeddings [14] capture local neighborhood connectivity. For more on the distinction between structural and proximity-preserving node embeddings, we refer the reader to [23].

**Network Alignment.** Classic graph alignment approaches often formulate an **optimization-based assignment** problem. For example, the message-passing algorithm NetAlign [3] tries to preserve “complete squares” by matching two nodes sharing an edge in one graph to counterparts sharing an edge in the other graph. FINAL [25] optimizes a topological consistency objective which may be augmented with node and edge attribute information. Our approach is initialized by the solution to a classic convex optimization formulation [9], but to improve the accuracy, we turn to a different class of methods: those that **compare node embeddings**.

REGAL [14] matches xNetMF structural embeddings that are comparable across networks. Subsequent work [7] models intra-network proximity via link prediction, but its cross-network comparison is also based on structural similarity. To use transductive proximity-preserving embedding objectives, workarounds include connecting the two graphs with ground truth “seed” alignments [19] if any are known, or using adversarial training techniques in machine translation [17] as in another recent work [6].

### 3 PRELIMINARIES

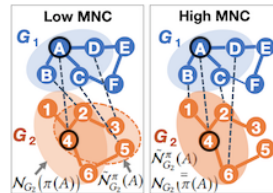
**Graphs and Embeddings.** We consider two graphs  $G_1$  and  $G_2$  with nodesets  $\mathcal{V}_1, \mathcal{V}_2$  and adjacency matrices  $A_1, A_2$  containing edges between nodes. As in [14], for simplicity, we assume that both graphs have  $n$  nodes (if not, we can add singleton nodes to one graph). For each graph  $G_i$ , we can create an  $n \times d$  matrix  $Y_i$  of  $d$ -dimensional node embeddings.

**Alignment.** An alignment between the nodes of two graphs is a function  $\pi : \mathcal{V}_1 \rightarrow \mathcal{V}_2$ , or alternatively a matrix  $P$ , where  $p_{ij}$  is the (real-valued or binary) similarity between node  $i$  in  $G_1$  and node  $j$  in  $G_2$ . A mapping  $\pi$  can be found from  $P$ , e.g. greedy alignment  $\pi(i) = \arg \max_j p_{ij}$ .

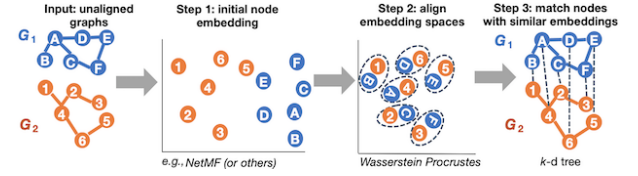
**Neighborhood.** Let  $\mathcal{N}_{G_1}(i)$  be the neighbors of node  $i$  in  $G_1$ , i.e., nodes that share an edge with  $i$ . We define node  $i$ ’s “**mapped neighborhood**” in  $G_2$  as the set of nodes onto which  $\pi$  maps  $i$ ’s neighbors:  $\tilde{\mathcal{N}}_{G_2}^\pi(i) = \{j \in \mathcal{V}_2 : \exists k \in \mathcal{N}_{G_1}(i) \text{ s.t. } \pi(k) = j\}$ . Also, we denote the neighbors of node  $i$ ’s counterpart  $\pi(i)$  as  $\mathcal{N}_{G_2}(\pi(i))$ . We define the **matched neighborhood consistency (MNC)** of node  $i$  in  $G_1$  and  $j$  in  $G_2$  as the Jaccard similarity of the two sets (visualized for a toy example in Fig. 1):

$$MNC(i, j) = \frac{|\tilde{\mathcal{N}}_{G_2}^\pi(i) \cap \mathcal{N}_{G_2}(j)|}{|\tilde{\mathcal{N}}_{G_2}^\pi(i) \cup \mathcal{N}_{G_2}(j)|} \quad (1)$$

**Problem Statement.** Given two graphs  $G_1$  and  $G_2$  with meaningful node correspondences, but none known *a priori*, we seek to recover their alignment  $\pi$  while achieving high MNC.



**Figure 1: Partial alignments between  $G_1$  &  $G_2$ : varying matched neighborhood consistency for node  $A$  in  $G_1$  and its counterpart in  $G_2$ , node 4.**



**Figure 2: Overview of CONE-Align.** Given two graphs  $G_1$  and  $G_2$ , we first use node embedding to model intra-graph node proximity. Second, we align the embedding spaces for cross-graph comparability. Third, we match each node in  $G_1$  to the node in  $G_2$  with the most similar embedding.

### 4 METHOD

We detail CONE-Align (Fig. 2, with pseudocode in Alg. 2), our proposed method using node embeddings to respect matched neighborhood consistency and to identify cross-graph node similarities.

#### 4.1 Step 1: Node Embedding

In Step 1, we obtain normalized node embeddings  $Y_1, Y_2 \in \mathbb{R}^{n \times d}$  separately per input graph. CONE-Align is a framework with which we can use many popular embedding methods, graph neural networks, etc. [10], *even though* they may be designed for a single network. We only need the embeddings to preserve intra-graph node proximity, i.e. neighboring nodes in each graph have similar embeddings and will be mapped close by when using embedding similarity. This preserves MNC *robustly*: even when nodes are not neighbors due to missing edges [7], many node embedding algorithms can preserve any higher-order proximities they share.

#### 4.2 Step 2: Embedding Space Alignment

Due to the invariance of the embedding objective, the two graphs’ node embeddings  $Y_1$  and  $Y_2$  may be translated, rotated, or rescaled relative to each other. Thus, to compare them, in Step 2, we align the embedding subspaces. Inspired by unsupervised word translation [11], we jointly solve two optimization problems:

**Procrustes.** If *node correspondences* were known, we could find a linear embedding transformation  $Q$  from the set of orthogonal matrices  $O^d$ .  $Q$  aligns the columns of the node embedding matrices, i.e. the embedding spaces. It can be obtained by solving an orthogonal Procrustes problem:

$$\min_{Q \in O^d} \|Y_1 Q - Y_2\|_2^2 \quad (\text{column permutation}) \quad (2)$$

Its solution is  $Q^* = UV^T$ , where  $U\Sigma V^T$  is the SVD of  $Y_1^T Y_2$  [24].

**Wasserstein.** If the *embedding space transformation* were known, we could solve for the optimal node correspondence  $P$  from the set of permutation matrices  $\mathcal{P}^n$ .  $P$  aligns the rows of the node embedding matrices, i.e. the nodes. It can be obtained using the Sinkhorn algorithm [5] to minimize the squared Wasserstein distance:

$$\min_{P \in \mathcal{P}^n} \|Y_1 - PY_2\|_2^2 \quad (\text{row permutation}) \quad (3)$$

**Wasserstein Procrustes.** As we know neither the *correspondences* nor the *transformation*, we combine the problems:

$$\min_{Q \in O^d} \min_{P \in \mathcal{P}^n} \|Y_1 Q - PY_2\|_2^2 \quad (4)$$

**Algorithm 1** Align Embeddings ( $Y_1, Y_2, A_1, A_2$ )

---

```

1: Input: node embeddings  $Y_1, Y_2$ , adjacency matrices  $A_1, A_2$ 
   /* Convex Initialization */
2:  $P^* = \arg \min_{P \in \mathcal{B}^n} \|(A_1 P - P A_2)\|_2^2$   ▷ Initial node correspondences:
   ▷ based on Franke-Wolfe [8] and Sinkhorn [5]
3:  $U \Sigma V^T = \text{SVD}(Y_1^T P^* Y_2)$ 
4:  $Q = UV^T$   ▷ Compute initial embedding space transformation
   /* Stochastic Alternating Optimization */
5: for  $t = 1 \rightarrow T$  do  ▷ T: # iter (e.g., 50)
6:    $P_t = \arg \max_{P \in \mathcal{B}^n} \text{trace}(Q^T Y_1^T P Y_2)$   ▷ Via Sinkhorn, compute
   ▷ optimal matching for size- $b$  (e.g., 10) minibatches  $Y_{1t}, Y_{2t}$ 
7:    $G_t = -2Y_{1t}^T P_t Y_{2t}$   ▷ Compute gradient of WP distance wrt  $Q$ 
8:    $U \Sigma V^T = \text{SVD}(Q - \eta G_t)$   ▷ Update orthogonal transform. matrix
9:    $Q = UV^T$   ▷  $\eta$ : learning rate (e.g., 1.0)
10: return  $Q$   ▷ orthogonal transformation  $Q$ 

```

---

**Algorithm 2** CONE-Align ( $A_1, A_2$ )

---

```

1: Input: adjacency matrices  $A_1, A_2$ 
   /* STEP 1. Model Intra-Network Proximities with Embeddings */
2:  $Y_1 = \text{proximity-emb-method}(A_1), Y_2 = \text{proximity-emb-method}(A_2)$ 
3:  $Y_1 = \frac{Y_1}{\|Y_1\|_2}, Y_2 = \frac{Y_2}{\|Y_2\|_2}$   ▷ Normalize the node embeddings
   /* STEP 2. Align Embedding Spaces for Cross-Graph Comparability */
4:  $Q = \text{Align Embeddings}(Y_1, Y_2, A_1, A_2)$ 
   /* STEP 3. Match Nodes with Similar Embeddings */
5:  $Y_1 = Y_1 Q$   ▷ Align embedding spaces and greedily match nodes with
6:  $P = \text{QueryKDTree}(Y_1, Y_2)$   ▷ sim. embeddings via  $k$ -d tree (NN search)
7: return  $P$   ▷ permutation matrix with aligned nodes across input graphs

```

---

We equivalently solve  $\max_{P \in \mathcal{P}^n} \max_{Q \in O^d} \text{trace}(Q^T Y_1^T P Y_2)$  with a stochastic optimization scheme [11], alternating between the Wasserstein and Procrustes problems. For  $T$  iterations, we use the current embedding transformation  $Q$  to find a matching  $P_t$  for minibatches  $Y_{1t}, Y_{2t}$  of  $b$  embeddings each, using Sinkhorn [5] with regularization parameter  $\lambda$ . We then use the gradient of the Wasserstein Procrustes distance  $\|Y_{1t} Q - P_t Y_{2t}\|_2^2$ , evaluated on the minibatches  $Y_{1t}, Y_{2t}$ , to update  $Q$  with gradient descent (Alg. 1).

**Convex Initialization.** To initialize the above *nonconvex* procedure, we turn to a classic convex graph matching formulation [9]:

$$\min_{P \in \mathcal{B}^n} \|(A_1 P - P A_2)\|_2^2 \quad (5)$$

where  $\mathcal{B}^n$  is the convex hull of  $\mathcal{P}^n$ . We can find the global minimizer  $P^*$  with the Frank-Wolfe algorithm [8] for  $n_0$  iterations and Sinkhorn [5] with regularization parameter  $\lambda_0$ . Using  $Y_1$  and  $P^* Y_2$ , an initial  $Q$  can be generated with orthogonal Procrustes (Eq. (2)).

**Complexity Considerations.** Our subspace alignment procedure (Alg. 1) uses SVD and Sinkhorn’s algorithm [5] on the full data. Although these algorithms have quadratic time complexity, recent superlinear approximations [1, 2] can further scale up CONE-Align.

### 4.3 Step 3: Matching Nodes with Embeddings

After aligning the embeddings with the final transformation  $Q$ , in Step 3, we match each node in  $G_1$  to its nearest neighbor in  $G_2$  based on Euclidean distance. We could use scaling corrections to mitigate “hubness” whereby many nodes have the same nearest

neighbor [11], but we found no need. Following [14], we use a  $k$ -d tree for fast nearest neighbor search between  $Y_1 Q$  and  $Y_2$ .

## 5 EXPERIMENTS

In this section, we analyze CONE-Align’s accuracy and matched neighborhood consistency in network alignment.

**Configuration of CONE-Align.** We use NetMF [21] node embeddings, which we find obtain higher accuracy than the related DeepWalk and node2vec [12, 20], possibly because the latter use random walks that increase variance [14]. We use default values [21], approximating the normalized graph Laplacian with 256 eigenpairs, and set embedding dimension  $d = 128$ , context window size  $w = 10$ , and  $\alpha = 1$  negative samples [21]. For the subspace alignment, we use parameters which yield good accuracy and speed:  $n_0 = 10$  iterations and regularization  $\lambda_0 = 1.0$  for the initial convex matching, and  $T = 50$  iterations of Wasserstein Procrustes optimization with batch size  $b = 10$ , learning rate  $\eta = 1.0$ , and regularization  $\lambda = 0.05$ .

**Data.** Following prior works [6, 14, 25], we simulate a network alignment scenario with known ground truth: a graph with adjacency matrix  $A$  is aligned to a noisy permuted copy  $A^*$ . We generate a random permutation matrix  $\bar{P}$  and set  $A^* = \bar{P} A \bar{P}^T$ ; we then randomly remove edges from  $A^*$  with probability  $p \in [0.05, 0.10, 0.15, 0.20, 0.25]$ . We perform this procedure on graphs representing various phenomena as shown in Table 1.

**Table 1: Description of the datasets used.**

| Name             | Nodes | Edges  | Description                 |
|------------------|-------|--------|-----------------------------|
| Arenas [16]      | 1 133 | 5 451  | communication network       |
| Hamsterster [16] | 2 426 | 16 613 | social network              |
| PPI [4]          | 3 890 | 76 584 | protein-protein interaction |
| Facebook [18]    | 4 039 | 88 234 | social network              |

**Baselines.** Our baselines are unsupervised methods using diverse techniques (belief propagation, spectral methods, and embeddings): (1) **NetAlign** [3], (2) **FINAL** [25], and (3) **REGAL** [14]. We configure each method following the literature. NetAlign and FINAL require a matrix of prior alignment information, for which we take the top  $k = \lfloor \log_2 n \rfloor$  most similar nodes by degree for each node [6, 14]. For REGAL we use recommended embedding dimension  $\lfloor 10 \log_2(2n) \rfloor$ , maximum neighbor distance 2 with discount factor  $\alpha = 0.1$ , and resolution parameter  $\gamma_{\text{struc}} = 1$  [14].

### 5.1 Alignment Performance

**5.1.1 Evaluation.** We measure **alignment accuracy**, or the proportion of correctly aligned nodes, as well as the average **matched neighborhood consistency (MNC)** using Eq. (1) across all nodes.

**5.1.2 Results.** In Fig. 3, we report average and standard deviation for each metric over five trials for each experimental setting.

**CONE-Align outperforms baselines.** We study  $5 \times$  higher noise levels than prior work [14]; in this challenging setting, NetAlign and FINAL achieve  $< 10\%$  accuracy. REGAL is most accurate on the PPI and Facebook networks with low noise; in these denser graphs, many nodes share first-order proximities and are hard to distinguish. However, CONE-Align outperforms it above  $10\%$  noise.

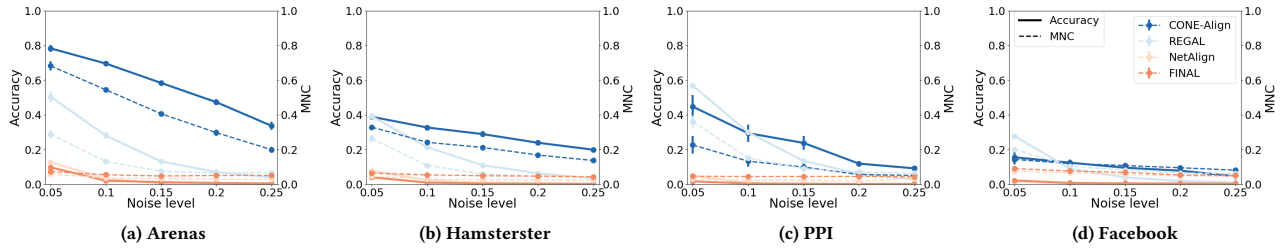


Figure 3: Average accuracy (solid lines) and MNC (dashed lines), with standard deviation in error bars, vs. different noise levels. CONE-Align significantly outperforms baselines and better preserves MNC across datasets, particularly as noise increases.

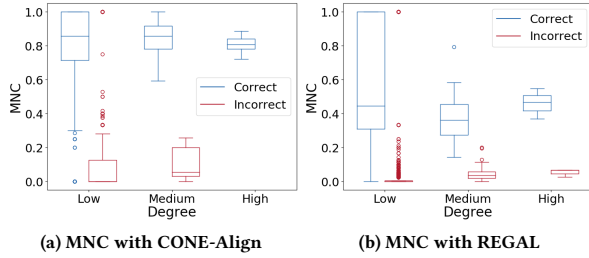


Figure 4: MNC of CONE-Align and REGAL on the Arenas dataset with 5% noise. Compared to REGAL, CONE-Align generates significantly higher MNC for almost all nodes.

**CONE-Align is more robust to noise.** CONE-Align’s accuracy declines less sharply than REGAL as noise increases, and it is the only method to measurably align any datasets at 25% noise.

**Accuracy and MNC are closely related.** They trend similarly, and more accurate methods (esp. CONE-Align) have higher MNC.

**Runtime.** CONE-Align’s average runtime per dataset ranges from 5 sec to 4 min: slower than the famously scalable methods NetAlign and REGAL, but at least twice as fast as FINAL.

## 5.2 Matched Neighborhood Consistency

To further understand MNC, we analyze it on a node-level basis.

**5.2.1 Setup & Evaluation.** For brevity, we show only REGAL and CONE-Align on the Arenas dataset with 5% noise. We split the nodes into three groups by degree:  $[0, \frac{\Delta^*}{3})$ ,  $[\frac{\Delta^*}{3}, \frac{2\Delta^*}{3})$ ,  $[\frac{2\Delta^*}{3}, \Delta^*]$ , where  $\Delta^*$  is the maximum degree, and plot the distribution of MNC for both correctly and incorrectly aligned nodes in Figure 4.

**5.2.2 Results.** For both methods, **MNC is much higher for correctly aligned nodes** across degree levels, but a few lower degree nodes may be misaligned with high MNC; their smaller neighborhoods may be misaligned together. However, CONE-Align correctly aligns all high-degree nodes.

## 6 CONCLUSION

CONE-Align’s success offers the following takeaway: the quest for cross-network embedding *comparability* should not neglect intra-network *proximity* information. With embedding subspace alignment, we obtain compatibility while capturing proximity. In the future, this may allow transductive node embeddings to improve other multi-network tasks such as graph classification, where off the shelf they are not applicable [13]. Other future work includes using embeddings that model node/edge attributes.

## ACKNOWLEDGEMENTS

We would like to thank the reviewers for their constructive feedback. This work is supported by the NSF under Grant No. IIS 1845491, Army Young Investigator Award No. W911NF1810397, and Adobe, Amazon, and Google faculty awards.

## REFERENCES

- [1] Zeyuan Allen-Zhu and Yuanzhi Li. Lazysvd: Even faster svd decomposition yet without agonizing pain. In *NeurIPS*, 2016.
- [2] Jason Altschuler, Francis Bach, Alessandro Rudi, and Jonathan Niles-Weed. Massively scalable sinkhorn distances via the nystrom method. In *NeurIPS*, 2019.
- [3] Mohsen Bayati, Margot Gerritsen, David F Gleich, Amin Saberi, and Ying Wang. Algorithms for large, sparse network alignment problems. In *ICDM*, 2009.
- [4] Bobby-Joe Breitkreutz, Chris Stark, et al. The biogrid interaction database: 2008 update. *Nucleic acids research*, 36(suppl 1):D637–D640, 2008.
- [5] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *NeurIPS*, 2013.
- [6] Tyler Derr, Hamid Karimi, Xiaorui Liu, Jiejun Xu, and Jiliang Tang. Deep adversarial network alignment. *arXiv preprint arXiv:1902.10307*, 2019.
- [7] Xingbo Du, Junchi Yan, and Hongyuan Zha. Joint link prediction and network alignment via cross-graph embedding. In *IJCAI*, 2019.
- [8] Marguerite Frank and Philip Wolfe. An algorithm for quadratic programming. *Naval research logistics quarterly*, 3(1-2):95–110, 1956.
- [9] Steven Gold and Anand Rangarajan. A graduated assignment algorithm for graph matching. *TPAMI*, 18(4):377–388, 1996.
- [10] Palash Goyal and Emilio Ferrara. Graph embedding techniques, applications, and performance: A survey. *Knowledge-Based Systems*, 151:78–94, 2018.
- [11] Edouard Grave, Armand Joulin, and Quentin Berthet. Unsupervised alignment of embeddings with wasserstein procrustes. In *AISTATS*, 2019.
- [12] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *KDD*, 2016.
- [13] Mark Heimann, Tara Safavi, and Danai Koutra. Distribution of node embeddings as multiresolution features for graphs. In *ICDM*, 2019.
- [14] Mark Heimann, Haoming Shen, Tara Safavi, and Danai Koutra. Regal: Representation learning-based graph alignment. In *CIKM*, 2018.
- [15] Ehsan Kazemi. Network alignment: Theory, algorithms, and applications. Technical report, EPFL, 2016.
- [16] Jérôme Kunegis. Konect: the koblenz network collection. In *WWW*, 2013.
- [17] Guillaume Lample, Alexis Conneau, Marc’Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Word translation without parallel data. In *ICLR*, 2018.
- [18] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. <http://snap.stanford.edu/data>, June 2014.
- [19] Li Liu, William K Cheung, Xin Li, and Lejian Liao. Aligning users across social networks using network embedding. In *IJCAI*, 2016.
- [20] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *KDD*, 2014.
- [21] Jiezhong Qiu, Yuxiao Dong, Hao Ma, Jian Li, Kuansan Wang, and Jie Tang. Network embedding as matrix factorization: Unifying deepwalk, line, pte, and node2vec. In *WSDM*, 2018.
- [22] Leonardo FR Ribeiro, Pedro HP Saverese, and Daniel R Figueiredo. struc2vec: Learning node representations from structural identity. In *KDD*, 2017.
- [23] Ryan A Rossi, Di Jin, Sungchul Kim, Nesreen K Ahmed, Danai Koutra, and John Boaz Lee. On proximity and structural role-based embeddings in networks: Misconceptions, methods, and applications. *TKDD*, 2020.
- [24] Peter H Schönemann. A generalized solution of the orthogonal procrustes problem. *Psychometrika*, 31(1):1–10, 1966.
- [25] Si Zhang and Hanghang Tong. Final: Fast attributed network alignment. In *KDD*, 2016.